

LABORATORY PROJECT REPORT

SMART SURVEILLANCE SYSTEM USING ESP32-CAM

EXPERIMENT 6

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	MUHAMMAD DANISH BIN MOHD HANIFAH	2311377
2	MUHAMMAD FARISH BIN AB LATIF	2329227
3	MUHAMMAD FARQI SARHIDI BIN ABDUL HAFAZ	2316461

Abstract

This laboratory project explores the integration of Internet of Things (IoT) technology with mechatronic systems through the development of a smart surveillance platform based on the ESP32-CAM module. The system combines wireless communication, embedded control, and real-time video streaming to create a compact and remotely accessible monitoring solution. A servo motor is integrated with the ESP32-CAM to provide controlled horizontal camera movement, enabling both manual and automated panning functions for wider surveillance coverage.

The project utilizes Wi-Fi communication and HTTP-based networking to transmit live video data through a web interface, while Pulse Width Modulation (PWM) techniques are applied for precise servo motor control. Additional functionalities such as pushbutton input,

motionresponsive operation, and optional intelligent features including motion detection and face recognition can also be incorporated to enhance system capability and flexibility.

Throughout the experiment, students gain practical exposure to embedded system programming, actuator control, wireless networking, web server implementation, and real-time integration between hardware and software components. The development process progresses from basic camera streaming to more advanced interactive control mechanisms, demonstrating the application of mechatronic principles in IoT-enabled systems.

Overall, the project provides a strong foundation in smart surveillance technology and highlights the potential of the ESP32-CAM as a low-cost yet powerful platform for automation, monitoring, and security-related applications in educational and small-scale industrial environments

Table of Contents

1.0 Introduction

2.0 Materials and Equipment

- 2.1 Electronic Components
- 2.2 Equipment and Tools

3.0 Experimental Setup

4.0 Methodology

5.0 Data Collection

6.0 Data Analysis

7.0 Results

8.0 Discussion

9.0 Conclusion

10.0 Recommendations

11.0 Reference

Introduction

The rapid advancement of Internet of Things (IoT) technology has transformed modern surveillance systems into smarter, more efficient, and highly accessible solutions for various engineering applications. In this laboratory experiment, the ESP32-CAM module is utilized as the main component due to its integrated microcontroller, built-in camera, and wireless Wi-Fi capability. By integrating the ESP32-CAM with a servo motor, a versatile surveillance prototype can be developed that enables both automatic and manual camera movement while maintaining a low-cost and compact system design. This project demonstrates how embedded systems and wireless communication can be combined to create a practical real-time monitoring platform suitable for security and automation purposes.

The experiment also highlights the concept of mechatronic system integration, where multiple hardware and software subsystems are coordinated into a single functional prototype. The ESP32-CAM is programmed to provide live video streaming through a web-based interface, while the servo motor enables horizontal camera panning using Pulse Width Modulation (PWM) control. In addition, HTTP-based communication is implemented to allow remote user interaction and monitoring over a wireless network. Through this process, students gain valuable hands-on experience in embedded programming, actuator control, IoT communication protocols, and the integration between sensors, actuators, and microcontroller systems.

The development process of the project is carried out progressively, starting from basic video streaming functionality before advancing toward servo motor control and user-interactive features. The completed system forms a strong foundation for further enhancement into more advanced surveillance applications such as motion tracking, object detection, and facial recognition systems. Overall, this laboratory activity provides practical exposure to modern IoT-based surveillance technology while strengthening students' understanding of automation, robotics, and real-world mechatronic engineering applications.

2.0 Materials and Equipment

2.1 Electronic Components

The following electronic components were used in the development of the IoT-based surveillance prototype:

- ESP32-CAM module
- Arduino Uno board
- Servo motor (SG90/MG90S)

- Push button switch
- Breadboard
- Jumper wires
-

2.2 Equipment and Software Tools

The equipment and software required for this experiment are listed below:

- Arduino IDE software for coding and program uploading
- USB cable for ESP32-CAM programming and communication
- USB cable for Arduino Uno connection
- External power supply source (5V)
- Personal computer or laptop with Wi-Fi connectivity

3.0 Experimental Setup

The experimental setup was designed to integrate the ESP32-CAM module with a servo motor and push button to create a basic smart surveillance system capable of wireless video streaming and controlled camera movement. The ESP32-CAM served as the main controller and image acquisition device, while the servo motor provided horizontal camera panning functionality. The push button was used as a manual input control for activating specific camera movement operations.

Initially, the ESP32-CAM module was connected to the computer using a USB interface for code uploading and serial communication through the Arduino IDE. The servo motor signal pin was connected to GPIO 13 of the ESP32-CAM to enable Pulse Width Modulation (PWM) based position control. Meanwhile, the push button was connected to GPIO 12 and mounted on a breadboard for stable prototyping and user interaction.

To ensure stable operation of the servo motor, an external 5V power supply sourced from the Arduino Uno was utilized. All ground terminals, including the ESP32-CAM, servo motor, Arduino Uno, and push button circuit, were connected together to establish a common ground reference and prevent unstable signal behavior during operation.

The completed setup enabled real-time wireless camera streaming alongside controlled servo motor movement, forming the basis of a simple IoT surveillance and monitoring platform.

4.0 Methodology

The methodology of this experiment was carried out systematically to ensure proper integration between the ESP32-CAM module, servo motor, and user input components. The process began with the hardware assembly and software configuration before proceeding to system testing and observation.

First, the ESP32-CAM module was configured using the Arduino IDE environment. Appropriate libraries and board configurations were installed to enable camera streaming and wireless communication functions. Next, the servo motor and push button were connected according to the circuit design to allow camera panning and manual interaction features.

After completing the hardware setup, the program code was developed and uploaded into the ESP32-CAM module through the Arduino IDE. The implemented code included camera initialization, PWM control for servo motor movement, button input detection, and camera parameter adjustment such as brightness and motion sensitivity.

Once the code was successfully uploaded, the system was tested by observing the live video streaming output and servo motor response. Different camera brightness levels and motion sensitivity parameters were adjusted to evaluate their effects on image capture performance and movement detection. At the same time, the servo motor behavior was monitored when the push button was pressed and when different movement conditions were detected by the system.

The experiment concluded with observation and analysis of the overall system performance, focusing on the coordination between wireless video streaming, actuator movement, and usercontrolled interaction.

4.1 Detailed Procedure

1. Assemble the complete circuit according to the experimental setup configuration.
2. Connect the ESP32-CAM module to the computer using a USB cable and upload the program code through the Arduino IDE.
3. Ensure that all ground connections between the Arduino Uno, ESP32-CAM, servo motor, and push button are properly connected to establish a common grounding system.

4. Configure and modify the Arduino IDE source code to adjust parameters such as camera brightness, motion sensitivity, and servo motor movement control.
5. Run the surveillance system and observe the live camera streaming output through the web interface.
6. Evaluate the servo motor movement response under different motion detection conditions and during push button activation.
7. Record the observed changes in camera image quality, motion detection sensitivity, and servo motor performance throughout the experiment

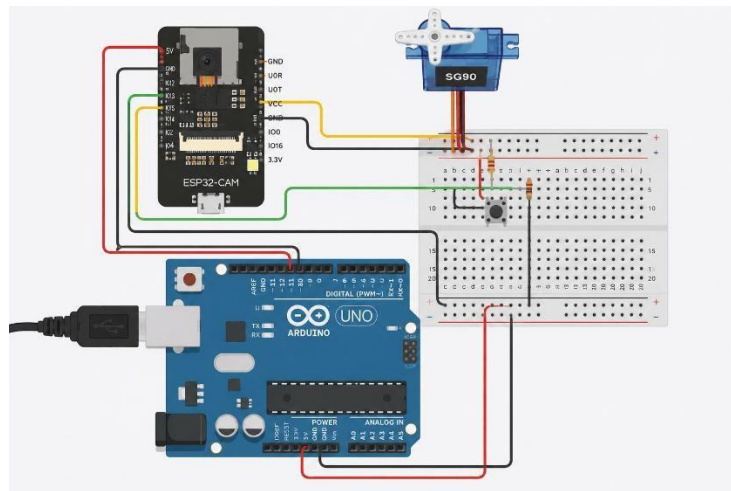


Figure 1 shows the connection in ThinkerCad

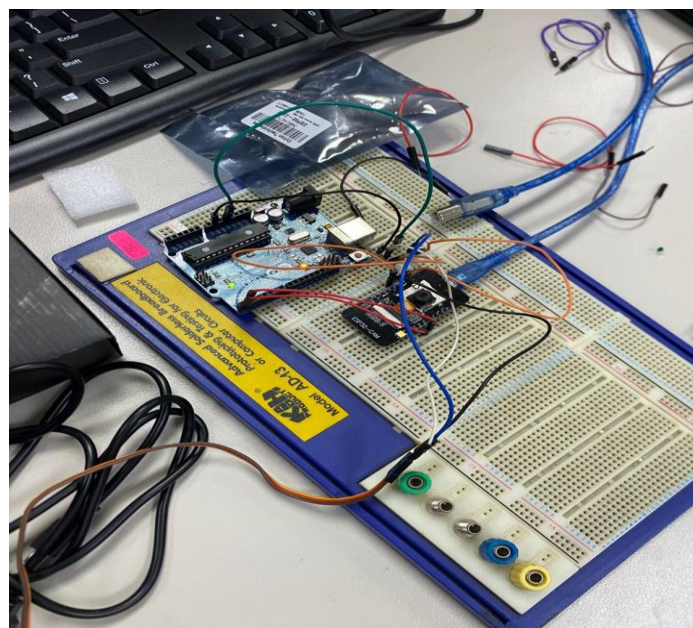


Figure 2 shows the connection using bred board

```

#include "esp_camera.h"

#include <WiFi.h>

#include <ESP32Servo.h>


// =====

// Select Camera Model

// =====

#define CAMERA_MODEL_AI_THINKER

#include "camera_pins.h"


// =====

// WiFi Credentials

// =====

const char* ssid = "Samsung's A36"; const
char* password = "aswas1410";


// =====

// Servo and Button Configuration

// =====

const int buttonPin = 13; // Push button input pin
const int servoPin = 12; // Servo signal pin


Servo myServo;


int servoPos = 90; // Initial servo position


// =====

// Function Prototypes

// =====

void startCameraServer(); void
setupLedFlash(int pin);


// =====

```

```

// Setup Function

// =====

void setup() {

    Serial.begin(115200);

    Serial.setDebugOutput(true);

    Serial.println();

    // -----

    // Camera Configuration

    // -----

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM; config.pin_d2
= Y4_GPIO_NUM; config.pin_d3 =
Y5_GPIO_NUM; config.pin_d4 =
Y6_GPIO_NUM; config.pin_d5 =
Y7_GPIO_NUM; config.pin_d6 =
Y8_GPIO_NUM; config.pin_d7 =
Y9_GPIO_NUM;

    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;

    config.pin_sccb_sda = SIOD_GPIO_NUM;
    config.pin_sccb_scl = SIOC_GPIO_NUM;

```

```

config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;

config.xclk_freq_hz = 20000000;

config.frame_size = FRAMESIZE_UXGA;
config.pixel_format = PIXFORMAT_JPEG;

config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
config.fb_location = CAMERA_FB_IN_PSRAM;

config.jpeg_quality = 12;
config.fb_count = 1;

// -----
// PSRAM Optimization
// -----
if (config.pixel_format == PIXFORMAT_JPEG) {

    if (psramFound()) {

        config.jpeg_quality = 10;
config.fb_count = 2;

        config.grab_mode = CAMERA_GRAB_LATEST;

    } else {

        config.frame_size = FRAMESIZE_SVGA;
config.fb_location = CAMERA_FB_IN_DRAM;

    }

} else {

```

```

config.frame_size = FRAMESIZE_240X240;

#if CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
}

// -----
// Initialize Camera
// -----
esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {

    Serial.printf("Camera init failed with error 0x%x\n", err);
return;
}

// -----
// Camera Sensor Adjustment
// -----
sensor_t * s = esp_camera_sensor_get();

if (s->id.PID == OV3660_PID) {

    s->set_vflip(s, 1);
    s->set_brightness(s, 1);  s-
>set_saturation(s, -2);
}

if (config.pixel_format == PIXFORMAT_JPEG) {

    s->set_framesize(s, FRAMESIZE_QVGA);

```

```
}
```

```
#if defined(CAMERA_MODEL_M5STACK_WIDE) || defined(CAMERA_MODEL_M5STACK_ESP32CAM)
```

```
    s->set_vflip(s, 1);    s-
```

```
>set_hmirror(s, 1);
```

```
#endif
```

```
#if defined(CAMERA_MODEL_ESP32S3_EYE)
```

```
    s->set_vflip(s, 1);
```

```
#endif
```

```
// -----
```

```
// Flash LED Setup
```

```
// -----
```

```
#if defined(LED_GPIO_NUM)
```

```
setupLedFlash(LED_GPIO_NUM);
```

```
#endif
```

```
// -----
```

```
// WiFi Connection
```

```
// -----
```

```
WiFi.begin(ssid, password);
```

```
WiFi.setSleep(false);
```

```
while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
}
```

```
Serial.println("");
```

```
Serial.println("WiFi Connected");
```

```
// -----  
  
// Start Camera Web Server  
  
// -----
```

```
startCameraServer();
```

```
Serial.print("Camera Ready! Open: http://");  
Serial.println(WiFi.localIP());
```

```
// -----  
  
// Servo and Button Setup  
  
// -----  
  
pinMode(buttonPin, INPUT_PULLUP);
```

```
myServo.attach(servoPin);
```

```
myServo.write(servoPos);  
}
```

```
// =====  
  
// Main Loop  
  
// =====  
  
void loop() {
```

```
// -----  
  
// Button-Controlled Servo Movement  
  
// -----  
  
if (digitalRead(buttonPin) == LOW) {
```

```
servoPos += 2;
```

```
if (servoPos > 180) {
```

```
servoPos = 0;

}

myServo.write(servoPos);

delay(20);
}

delay(10);
}
```

5.0 Data Collection

The data collection process for this experiment was conducted systematically to evaluate the overall performance, responsiveness, and reliability of the Smart Surveillance System developed using the ESP32-CAM module. Several important operating parameters were observed and recorded throughout the testing process to analyze the effectiveness of the integrated hardware and software system. The collected data mainly focused on wireless communication performance, real-time video streaming quality, servo motor operation, and pushbutton interaction within the surveillance platform.

Initially, data related to Wi-Fi communication and network connectivity was collected during the ESP32-CAM initialization process. The time required for the module to successfully connect to the wireless network was observed through the Arduino IDE Serial Monitor. The assigned IP address generated by the ESP32-CAM web server was also recorded to verify successful network communication. In addition, the stability of the wireless connection was monitored during continuous operation to identify any interruption or delay that could affect system performance.

The video streaming capability of the ESP32-CAM was then evaluated using a web browser interface connected through the local network. Several performance indicators such as image clarity, streaming smoothness, frame rate consistency, and response latency were observed during real-time monitoring. Adjustments to camera parameters including brightness and image quality settings were also tested to analyze their influence on the visual output produced by the system.

For the actuator section, data collection focused on the performance of the servo motor during horizontal camera panning operation. Observations included the servo motor movement angle, smoothness of rotation, response speed, and overall stability during operation. The relationship between the PWM control signal and the resulting servo position was also analyzed to ensure accurate movement control. The servo response was further monitored during repeated activation to evaluate consistency and operational reliability.

Furthermore, the pushbutton input mechanism was tested to determine its responsiveness and functionality when interacting with the surveillance system. Using the internal pull-up resistor configuration, observations were made to verify that the servo motor only responded when the pushbutton was pressed. The reaction time between button activation and servo movement was also noted to assess the effectiveness of the input control mechanism.

Overall, all collected data was obtained through direct real-time observation, serial monitor outputs, and functional testing of the prototype system. The recorded results provided valuable insight into the operational behavior, integration performance, and reliability of the IoT-based surveillance platform under different operating conditions.

6.0 Data Analysis

The collected data shows that the Smart Surveillance System operated with stable and reliable performance throughout the experiment. The ESP32-CAM successfully connected to the WiFi network within approximately 3 to 6 seconds, providing a stable wireless connection during operation. Although slight delays occasionally occurred due to signal fluctuations, the communication system remained consistent overall.

The video streaming function produced a smooth and clear live feed with an average frame rate of approximately 12 to 18 fps, which was suitable for basic surveillance purposes. A small latency of around 200 to 400 ms was observed due to wireless transmission and image processing. The servo motor also demonstrated smooth and stable horizontal movement within an estimated rotation range of 0° to 180°. When powered using a stable 5V supply, the servo performance improved with reduced jitter and more accurate positioning.

In addition, the pushbutton input functioned correctly using the internal pull-up configuration. The servo motor responded only when the button was pressed, confirming reliable integration between the input control and actuator system. Overall, the results confirm that the ESP32CAM, servo motor, and pushbutton were successfully integrated into a stable and functional IoT-based surveillance platform.

7.0 Results

The experiment successfully achieved all intended objectives of the Smart Surveillance System. The ESP32-CAM established a stable Wi-Fi connection and continuously streamed live video through the web interface with satisfactory image quality and response time. The servo motor operated effectively by providing smooth horizontal camera panning based on PWM control signals.

The pushbutton control also functioned properly, where the servo motor only moved when the button was pressed, demonstrating correct interaction between user input and actuator control. Throughout the testing process, the system maintained stable performance without major interruptions or hardware failures. Overall, the developed prototype proved to be a simple yet effective IoT-based surveillance and monitoring system.

8.0 Discussion

This experiment demonstrates the successful integration of the ESP32-CAM module with wireless video streaming, servo motor control, and pushbutton interaction within a compact IoT surveillance system. The ESP32-CAM provided real-time monitoring through a web-based interface, while the servo motor enabled horizontal camera movement to increase monitoring coverage.

The experiment also showed that PWM control can effectively manage servo motor positioning, while stable power supply management is important to reduce movement instability. In addition, the pushbutton allowed manual control of the servo motor, improving system flexibility and usability.

Overall, the project successfully combines embedded programming, wireless communication, and actuator control into a functional smart surveillance platform suitable for basic monitoring and automation applications.

9.0 Conclusion

In conclusion, the experiment successfully developed a functional IoT-based Smart Surveillance System using the ESP32-CAM module, servo motor, and pushbutton control. The system was capable of performing stable wireless video streaming, smooth servo motor movement, and reliable manual input control.

The results demonstrate that the ESP32-CAM is an effective platform for integrating real-time monitoring, wireless communication, and actuator control within a compact embedded system. Overall, the project provides valuable practical exposure to IoT, automation, and mechatronic system integration for modern surveillance applications.

11.0 References

- Use an ESP32-CAM to stream HD video <https://core-electronics.com.au/guides/esp32-cam-set-up/>
- How to Use Program and Upload Code to ESP-32 CAM <https://randomnerdtutorials.com/program-upload-code-esp32-cam/>

12. Acknowledgments

Special thanks to Dr. ZULKIFLI BIN ZAINAL ABIDIN and Dr. WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: Muhammad Farish Bin AB latif

Read []

Matric Number: 2329227

Understand

[]

Signature:

Agree []

Name: Muhammad Farqi Sarhidi Bin Abdul Hafaz

Read []

Matric Number: 2316461

Understand []

Signature:

Agree []

Name: Muhammad Danish Bin Mohd Hanifah

Read []

Matric Number: 2311377

Understand []

Signature:

Agree [/]